Braillix - Electronics Plan

Written 2026-08-01. This is the electronics source of truth. Scope: **electronics only.** Mechanical/CAD issues are *flagged* here, never fixed here.

Written for a CS student with basic soldering. Every "why" is answered, because several older documents in this repo tell you to build things you do not need.

PART 0 - The short version

Question	Answer
Do we need a custom PCB?	**No.** Not for one cell, and not for multi-cell either.
Do we need custom ICs?	**No.** The ATmega328P plan was over-engineering.
What do I own that works?	ESP32, 28BYJ-48 motor, ULN2003 driver, hall module, 5V adapter. **That is a complete single cell.**
What do I solder today?	Two wires on the DC jack. That is genuinely all.
What is the real blocker?	Nothing electronic. **The circuit has never been built on a breadboard.**

PART 1 - Why there is no custom PCB, and never was a good reason for one

The old plan called for a "muscle board": a custom PCB carrying an **ATmega328P** per cell, talking to the ESP32 over I2C.

Kill it. Three independent reasons:

1. **You cannot build it.** ATmega328P-AU is **TQFP-32 at 0.8mm pin pitch**. That is a

reflow-oven or hot-air part. The project's own docs already concede it must be ordered pre-assembled - which directly contradicts doing everything yourselves.

1. **It solves a problem you do not have.** Its job was to let many cells share a bus. You

have **one** cell. A single ESP32 drives it directly, today, with parts already in hand.

1. **When you *do* want many cells, an off-the-shelf module does the same job.** See Part 5.

> **Seven documents still reference the muscle board / KiCad / ATmega:** `DEMO_VS_PRODUCT.md`, `ELECTRONICS_BOM.md`, `MASTER_BOM.md`, `PRINT_CHECKLIST.md`, `SHOPPING_LIST.md`, `SOFTWARE_TEAM_README.md`, `WIRING_AND_ASSEMBLY.md`. Treat every one of those passages as **stale**. This file supersedes them.

"Why did we need any of this at all?"

Walk the chain once and it stops feeling arbitrary:

```
An ESP32 pin can supply about 20mA at 3.3V.
A 28BYJ-48 motor coil wants about 200mA at 5V.
                      -> connect them directly and you destroy the ESP32 pin.

So you need something that takes a small signal and switches a big current.
That is ALL the ULN2003 is. Seven electronic switches in one chip.
You already own it, on the little blue board with four red LEDs.
```

That is the entire reason any driver exists. Nothing custom is involved.

PART 2 - What lives where

```
BRAIN POD  (one, ever)              MUSCLE CELL  (one per character)
+--------------------------+        +---------------------------------+
|  ESP32 DevKit            |        |  28BYJ-48 stepper motor         |
|  3 nav buttons           |        |  Hall sensor (bare TO-92)       |
|  DC power jack           |        |  ULN2003 driver board           |
|  (multi-cell only:       |        |                                 |
|   2x 4.7k I2C pull-ups)  |        |  (multi-cell only: shares an    |
```

```
|                          |         |   MCP23017 with its neighbours) |
+--------------------------+         +---------------------------------+
          |                                        ^
          +---- 4 wires: 5V, GND, SDA, SCL ----------+
                (today: 6 loose wires instead)
```

Brain pod = decides *what* to show. **Muscle cell** = makes it physically happen.

PART 3 - [!!] Pin conflict, present in the code today

```
ASSEMBLY_BIBLE.md      GPIO21 -> ULN2003 IN3     GPIO22 -> ULN2003 IN4
SOFTWARE_TEAM_README   GPIO21 = I2C SDA          GPIO22 = I2C SCL
breadboard_test.ino    #define IN3 21            #define IN4 22
```

The motor and the I2C bus are assigned **the same two pins**.

- **Single cell:** harmless. No I2C exists yet. Do not change anything mid-demo.
- **The moment a second cell appears:** they collide, and neither works.

Fix when multi-cell starts - move the motor off the I2C pins:

Signal	Now	Change to	Why
IN1	18	18	fine
IN2	19	19	fine
IN3	**21**	**23**	frees SDA
IN4	**22**	**5**	frees SCL
Hall AO	34	34	ADC1, input-only, fine

Requires editing `breadboard_test.ino` and re-flashing. **Do not do this before the demo.**

PART 4 - Space: exactly one real constraint

Location	Space	Verdict
Cell electronics pocket	**36 x 46 x 16mm**	[!] **The only tight spot**
Pod interior	~38mm spare headroom	Fine, wildly oversized

The problem: a ULN2003 board with Dupont jumpers plugged in vertically stands **~20mm**. The pocket is **16mm**. It does not fit.

The fix: cut the Dupont ends off and solder the wires **flat** against the board. That drops it to ~12mm, leaving 4mm spare. This is why soldering appears at all.

Also flagged (CAD, not fixed here): the *whole blue hall module* does not fit in the base plate. Only the bare 3-legged sensor does, desoldered from the module, with three wires run back. **Not needed for the breadboard demo.**

PART 5 - Multi-cell, without a single custom part

Only relevant after one cell physically works. Recorded so nobody re-invents the PCB.

The problem: each cell needs 4 motor lines + 1 sensor = 5 pins. An ESP32 runs out at roughly 4 cells, and you cannot route 5 wires per cell between snap-together bricks.

The solution: an **I2C I/O expander** - a chip that gives you many pins over two shared wires, with a selectable address so several can share one bus.

Use MCP23017. Do NOT use PCF8574.

	MCP23017	PCF8574
I/O	16	8
Output type	**true push-pull**	quasi-bidirectional, **weak high side**
Drives a ULN2003 input?	**Yes**	[!] **Marginal - avoid**
Package	DIP-28 available, hand-solderable	DIP-16

	MCP23017	PCF8574
Addresses	8 (0x20-0x27)	8

Why this matters: a ULN2003 input needs a couple of mA *sourced into* it. The PCF8574's high state is a weak pull-up of roughly 100µA. It looks like it should work, and it will half-work - the worst possible failure. MCP23017 sources properly.

Per-cell budget on one MCP23017: 4 lines to the ULN2003, 1 line from the hall sensor's **DO** pin (digital, not AO), leaving 11 spare -> comfortably **3 cells per expander**, and 8 expanders per bus.

Total custom silicon required: zero. Total SMD soldering: zero.

PART 6 - What to actually do, in order

1. [!!] Check the jack polarity. Before anything else, ever.

There is **no reverse-polarity protection anywhere in this circuit.** Backwards = a dead ESP32, instantly and silently. Red-wire-means-positive is a convention, not a guarantee.

```
Multimeter on DC volts. Adapter in the wall. Nothing else connected.
  black probe on one wire, red probe on the other
  reads about +5V  -> the wire under the RED probe is POSITIVE
  reads about -5V  -> it is the other way round
Mark the positive wire. Nail polish, marker, anything.
```

Needs: a multimeter (~?500, still to buy).

2. The one soldering job available today

Your DC jack is an **inline pigtail** - a barrel socket with two bare wire ends and no thread or nut. Bare wires do not sit in a breadboard.

> **Tin both wire ends, then solder them to a 2-pin male header strip.** Now the adapter plugs into the breadboard like any other component.

Nothing is powered while you do this, so it is a safe first job. **Needs:** 0.8mm 60/40 rosin-core solder (~?150). Iron already owned.

*(Mechanical mounting of this jack in the pod lid is a **CAD** question - flagged, not solved here.)*

3. Build the breadboard circuit - the actual blocker

No soldering. Dupont jumpers only. Full wiring in `docs/ASSEMBLY_BIBLE.md`.

```
[5V adapter] --+--> breadboard 5V rail  --> ULN2003 (+)
               +--> breadboard GND rail --> ULN2003 (-)

ESP32 (powered by USB from the laptop)
  GND    --> breadboard GND rail       <-- COMMON GROUND. Without it the hall reads garbage.
  3V3    --> Hall VCC
  GND    --> Hall GND
  GPIO34 <-- Hall AO
  GPIO18 --> ULN2003 IN1
  GPIO19 --> ULN2003 IN2
  GPIO21 --> ULN2003 IN3
  GPIO22 --> ULN2003 IN4
```

Two rules that protect the hardware:

1. **Hall sensor VCC goes to 3V3, never 5V.** ESP32 GPIOs are not 5V tolerant.
2. **Never feed the adapter into ESP32** `VIN` **while USB is plugged into the laptop.**

Motor from the adapter, ESP32 from USB, **grounds joined only.**

4. Flash and confirm

`firmware/breadboard_test/breadboard_test.ino` -> motor turns, hall finds home, dashboard appears on your phone. **That is a legitimate, demonstrable result** - and it is the same circuit the first real cell uses, so none of it is throwaway.

PART 7 - Known electronics defect, not yet fixed

`breadboard_test.ino` **targets the wrong step position.**

```
long targetStep = (long)pos * STEPS_PER_REV / 64;        // lands mid-RAMP
long targetStep = (long)pos * STEPS_PER_REV / 64 + 32;   // lands mid-DWELL
```

The cam's ramps are centred on slice boundaries, so `pos x 64` stops the follower **on the slope** instead of on the flat. Verified numerically: at position 0 all six dots sit at factor 0.5 - half-raised, unreadable.

Not applied yet because it is tied to a mechanical re-derivation happening in the CAD fork. Recorded here so that if the bench shows half-raised dots, nobody hunts for a mechanical cause that does not exist.

PART 8 - Shopping, electronics only

Item	Why	?
Multimeter	[!!] Mandatory. The only thing standing between you and a dead ESP32.	~500
Breadboard	The entire demo runs on it	~100
Solder, 0.8mm 60/40 rosin core	The jack job, later the ULN2003	~150
Wire strippers	Prepping wire ends	~200
USB-C data cable	To flash the ESP32. Charge-only cables look identical and fail silently - **test yours first.**	~150

~?1,100, or ~?950 if your existing cable carries data.

Not needed: any driver IC (you own it), resistors, capacitors, flyback diodes (inside the ULN2003), level shifters, crystals, and **any custom PCB**.

PART 9 - Flagged for the CAD fork (do not fix here)

1. **Hall module does not fit** the base plate - needs the bare TO-92 desoldered, and a pocket that can hold it.
1. **DC jack has no thread or nut** - the pod lid cannot clamp it. Needs either a mechanical retainer in the box or a different jack.
1. **ULN2003 pocket is 16mm** - drives the solder-flat requirement. Working, but zero margin.